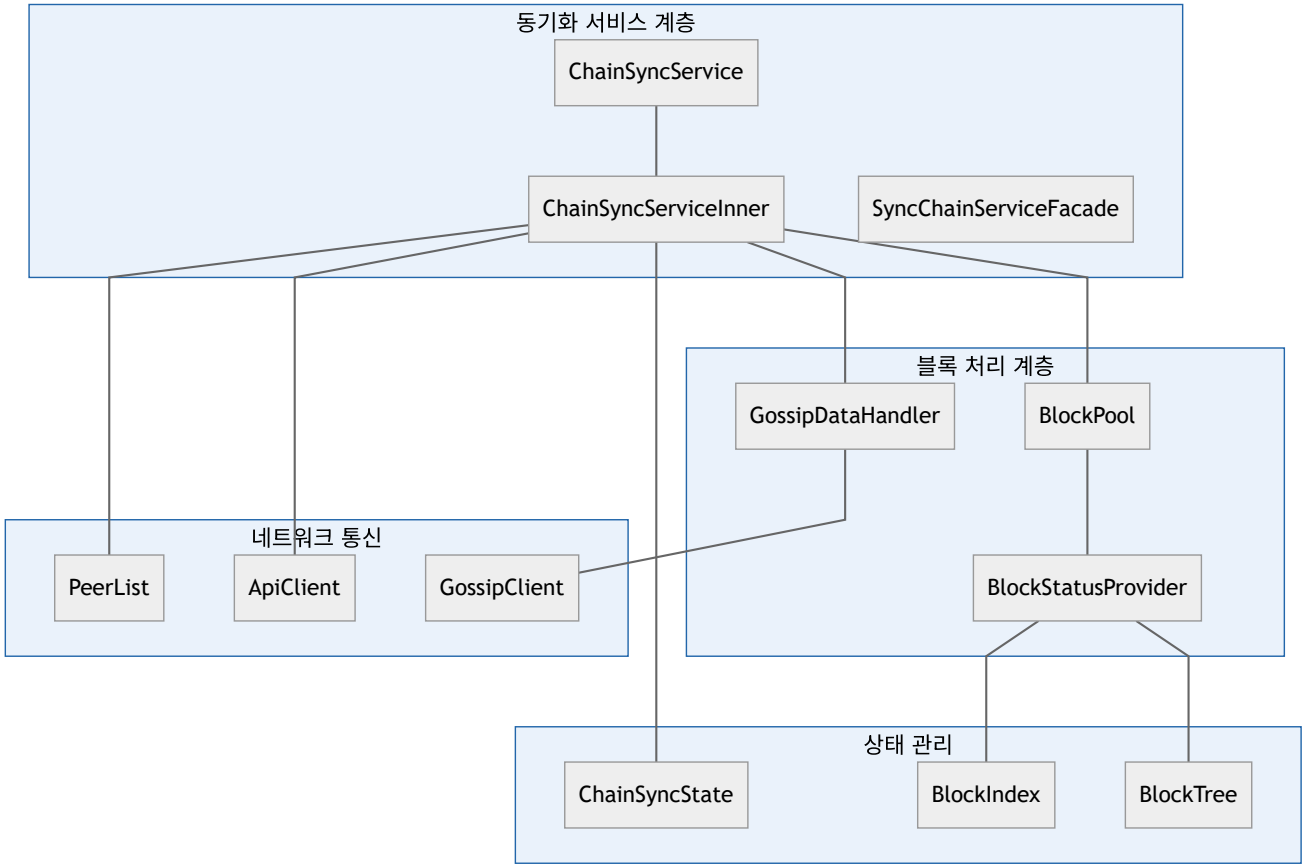
체인 동기화 및 블록 전파 시스템

개요

Irys 체인의 동기화 시스템은 분산 네트워크에서 노드들이 일관된 블록체인 상태를 유지하도록 하는 핵심 메커니즘입니다. 이 시스템은 초기 동기화, 지속적인 블록 전파, 포크 해결, 그리고 고아 블록 처리를 통해 네트워크의 일관성과 안정성을 보장합니다.

동기화 시스템 아키텍처



동기화 상태 관리

1. ChainSyncState 구조

```
pub  struct ChainSyncState {

/// 현재 동기화 중인지 여부

is_syncing: Arc<AtomicBool>,

/// 동기화 시작 높이

syncing_from: Arc<AtomicUsize>,
```

```
/// 동기화 대상 높이
sync_target_height: Arc<AtomicUsize>,
/// 처리된 최고 블록 높이
highest_processed_block: Arc<AtomicUsize>,
/// 신뢰할 수 있는 피어에서 동기화 중인지
is_trusted_sync: Arc<AtomicBool>,
/// 가십 수신 활성화 여부
gossip_reception_enabled: Arc<AtomicBool>,
/// 가십 브로드캐스트 활성화 여부
gossip_broadcast_enabled: Arc<AtomicBool>,
}
```

2. 동기화 모드

2.1 Genesis 모드

```
let  is_in_genesis_mode = matches!(node_mode, NodeMode::Genesis);

if  is_in_genesis_mode && sync_state.sync_target_height() <= 1 {
debug!("Genesis node with no blocks, skipping sync");
sync_state.finish_sync();
return Ok(());
}
```

2.2 신뢰할 수 있는 피어 동기화

```
let is_trusted_mode = matches!(node_mode, NodeMode::TrustedPeerSync);

if is_trusted_mode {
sync_state.set_trusted_sync(true);
let trusted_peers = peer_list.trusted_peers();
if trusted_peers.is_empty() {
return Err(ChainSyncError::Network("No trusted peers available".to_string()));
}
}
```

블록 인덱스 및 동기화 프로세스

1. 블록 인덱스 가져오기

```
async fn get_block_index(
peer_list: &PeerList,
api_client: &impl ApiClient,
start_height: usize,
batch_size: usize,
max_retries: usize,
fetch_from_trusted_peer: bool,
) -> ChainSyncResult<Vec<BlockIndexItem>> {
```

```
let mut peers = if fetch_from_trusted_peer {
peer_list.trusted_peers()
} else {
peer_list.top_active_peers(Some(10), None)
};


peers.shuffle(&mut rand::thread_rng());
peers.truncate(5);


for attempt in 1..=max_retries {
for peer in &peers {
match api_client
.get_block_index(
peer.1.address.api,
BlockIndexQuery {
start_height,
limit: batch_size,
},
)
.await
{
Ok(index) => return Ok(index),
Err(err) => {
```

```
warn!("Failed to get block index from peer {}: {}", peer.0, err);
continue;
}
}
}
if attempt < max_retries {
tokio::time::sleep(Duration::from_millis(1000)).await;
}
}

Err(ChainSyncError::Network("Failed to fetch block index".to_string()))
}
```

2. 배치 동기화 프로세스

```
const BLOCK_BATCH_SIZE: usize = 10;

async fn sync_chain<B, M, A>(
sync_state: ChainSyncState,
api_client: impl ApiClient,
peer_list: &PeerList,
mut start_sync_from_height: usize,
config: &Config,
```

```
gossip_data_handler: Arc<GossipDataHandler<M, B, A>>,
) -> ChainSyncResult<()> {
// 블록 인덱스 가져오기
let block_index = get_block_index(
peer_list,
&api_client,
sync_state.sync_target_height(),
BLOCK_BATCH_SIZE,
5,
fetch_index_from_trusted_peer,
).await?;

let mut target = sync_state.sync_target_height() + block_index.len();
let mut blocks_to_request = block_index.len();
let mut block_queue = VecDeque::new();
block_queue.extend(block_index);

// 블록 큐 처리
while let Some(block) = block_queue.pop_front() {
// 큐가 가득 찬 경우 대기
if sync_state.is_queue_full() {
sync_state.wait_for_an_empty_queue_slot().await?;
}
```

```
// 블록 요청 및 처리
let  block_hash = block.block_hash;
tokio::spawn(async  move {
sync_state_clone.increment_sync_target_height();
match  data_handler
.pull_and_process_block(
block_hash,
sync_state_clone.is_syncing_from_a_trusted_peer(),
)
.await
{
Ok(()) => {
debug!("Successfully processed block {:?}", block_hash);
}
Err(e) => {
error!("Failed to process block {:?}: {}", block_hash, e);
}
}
});

blocks_to_request -= 1;
// 추가 블록 인덱스 요청
```

```
if  blocks_to_request == 0 {
let  additional_index = get_block_index(
peer_list,
&api_client,
target,
BLOCK_BATCH_SIZE,
5,
fetch_index_from_trusted_peer,
).await?;


target += additional_index.len();
block_queue.extend(additional_index);
blocks_to_request = block_queue.len();
if  blocks_to_request == 0 {
break;
}
}
}


// 동기화 완료 대기
sync_state.wait_for_processed_block_to_reach_target().await;
sync_state.finish_sync();
Ok(())
```

```
}
```

블록 풀 관리 시스템

1. 블록 풀 구조

```
pub  struct BlockPool<B, M>
where
B: BlockDiscoveryFacade,
M: MempoolFacade,
{
/// 데이터베이스 제공자
db: DatabaseProvider,
/// 블록 캐시
blocks_cache: BlockCacheGuard,
/// 블록 발견 서비스
block_discovery: B,
/// 멤풀 서비스
mempool: M,
/// 동기화 서비스 송신자
sync_service_sender: mpsc::UnboundedSender<SyncChainServiceMessage>,
/// 동기화 상태
sync_state: ChainSyncState,
```

```
/// 블록 상태 제공자
block_status_provider: BlockStatusProvider,
/// 실행 페이로드 제공자
execution_payload_provider: ExecutionPayloadCache,
}
```

2. 블록 처리 워크플로우

```
pub  async  fn  process_block(
&self,
block_header: Arc<IrysBlockHeader>,
skip_validation_for_fast_track: bool,
) -> Result<(), BlockPoolError> {
// 블록 상태 확인
check_block_status(
&self.block_status_provider,
block_header.block_hash,
block_header.height,
)?;

// 중복 처리 방지
let  is_processing = self.blocks_cache.is_block_processing(&block_header.block_h
if  is_processing {
```

```
return Err(BlockPoolError::AlreadyProcessed(block_header.block_hash));
}

// 캐시에 블록 추가
self.blocks_cache.add_block(Arc::clone(&block_header), skip_validation_for_fast_

let prev_block_hash = block_header.previous_block_hash;
let current_block_hash = block_header.block_hash;

// 부모 블록 상태 확인
let previous_block_status = self
.block_status_provider
.block_status(block_header.height.saturating_sub(1), &prev_block_hash);

if !previous_block_status.is_processed() {
// 부모 블록이 없는 경우 고아 블록으로 처리
self.blocks_cache.change_block_processing_status(block_header.block_hash, false)
// 동기화 서비스에 부모 블록 요청
if let Err(send_err) = self.sync_service_sender.send(
SyncChainServiceMessage::RequestBlockFromTheNetwork {
block_hash: prev_block_hash,
response: None,
}
```

```
) {
error!("Failed to send RequestBlockFromTheNetwork message: {:?}", send_err);
}
return Ok(());
}

// 신뢰할 수 있는 동기화의 경우 실행 페이로드 미리 처리
if  skip_validation_for_fast_track {
self.handle_execution_payload_for_prevalidated_block(
block_header.evm_block_hash,
true
);
}

// 블록 트리 용량 대기
self.block_status_provider
.wait_for_block_tree_can_process_height(block_header.height)
.await;

// 블록 검증 및 처리
if  let Err(block_discovery_error) = self
.block_discovery
.handle_block(Arc::clone(&block_header), skip_validation_for_fast_track)
```

```
.await
{
error!("Block validation error: {:?}", block_discovery_error);
self.blocks_cache.remove_block(&block_header.block_hash).await;
return Err(BlockPoolError::BlockError(format!("{:?}", block_discovery_error)));
}

// 실행 페이로드 처리
if !skip_validation_for_fast_track {
self.handle_execution_payload_for_prevalidated_block(
block_header.evm_block_hash,
false,
);
}

// 처리 완료 표시
self.sync_state.mark_processed(block_header.height as usize);
self.blocks_cache.remove_block(&block_header.block_hash).await;

// 고아 블록 처리 알림
if  let Err(send_err) = self.sync_service_sender.send(
SyncChainServiceMessage::BlockProcessedByThePool {
block_hash: current_block_hash,
```

```
response: None,
}
) {
error!("Failed to send BlockProcessedByThePool message: {:?}", send_err);
}

Ok(())
}
```

3. 블록 상태 관리

```
#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
pub  enum BlockStatus {
/// 블록이 인덱스나 트리에 없음
NotProcessed,
/// 블록이 트리에 있지만 재구성 가능
ProcessedButCanBeReorganized,
/// 블록이 인덱스에 있고 최종 확정됨
Finalized,
}

impl BlockStatusProvider {
```

```
pub fn block_status(&self, block_height: u64, block_hash: &BlockHash) -> Block
let block_is_in_tree = self.is_block_in_the_tree(block_hash);
let binding = self.block_index_read_guard.read();
let index_item = binding.get_item(block_height);
let hash_is_in_index = index_item.is_some_and(|idx| idx.block_hash == *block_ha
let height_is_in_index = index_item.is_some();

if height_is_in_index {
if hash_is_in_index {
return BlockStatus::Finalized;
} else {
return BlockStatus::Finalized; // 포크 블록은 처리 불가
}
}

if !height_is_in_index && block_is_in_tree {
BlockStatus::ProcessedButCanBeReorganized
} else {
BlockStatus::NotProcessed
}
}
}
```

고아 블록 처리 시스템

1. 고아 블록 캐시

```
#[derive(Clone, Debug)]
struct BlockCacheInner {
/// 부모 해시별 고아 블록 캐시
pub orphaned_blocks_by_parent: LruCache<BlockHash, CachedBlock>,
/// 블록 해시에서 부모 해시로의 매핑
pub block_hash_to_parent_hash: LruCache<BlockHash, BlockHash>,
/// 요청된 블록들의 집합
pub requested_blocks: HashSet<BlockHash>,
}
```

2. 고아 블록 처리 워크플로우

```
/// 고아 블록의 조상 처리
async fn process_orphaned_ancestor(&self, block_hash: BlockHash) -> Result<(),
let maybe_orphaned_block = self
.block_pool
.get_orphaned_block_by_parent(&block_hash)
.await;

if let Some(orphaned_block) = maybe_orphaned_block {
```

```
info!("Processing orphaned ancestor block: {:?}", orphaned_block.header.block_ha


self.block_pool
.process_block(orphaned_block.header, orphaned_block.is_fast_tracking)
.await
.map_err(|e| ChainSyncError::Internal(format!("Block processing error: {:?}", e)
} else {
info!("No orphaned ancestor block found for: {:?}", block_hash);
Ok(())
}
}


/// 네트워크에서 부모 블록 요청
async fn request_parent_block(&self, parent_block_hash: BlockHash) -> Result<(
let parent_is_in_cache = self
.block_pool
.is_parent_hash_in_cache(&parent_block_hash)
.await;


if !parent_is_in_cache {
debug!("Parent block {:?} not in cache, requesting from network", parent_block_h
self.request_block_from_the_network(parent_block_hash).await
} else {
```

```
debug!("Parent block {:?} already in cache", parent_block_hash);

Ok(())

}

}
```

포크 해결 및 체인 재구성

1. 포크 감지

```
pub fn block_status(&self, block_height: u64, block_hash: &BlockHash) -> Block

let binding = self.block_index_read_guard.read();

let index_item = binding.get_item(block_height);

let hash_is_in_index = index_item.is_some_and(|idx| idx.block_hash == *block_ha

if index_item.is_some() && !hash_is_in_index {

// 같은 높이에 다른 블록이 있음 - 포크 상황

return BlockStatus::Finalized; // 마이그레이션된 높이의 포크는 처리 불가

}

// 정상적인 블록 상태 반환

}
```

2. 체인 재구성 메커니즘

```rust
/// 블록 트리에서 재구성 가능한 블록들 관리
impl BlockTree {
pub fn can_process_height(&self, block_height: u64) -> bool {
// 블록 트리가 해당 높이를 처리할 수 있는지 확인
let latest_canonical_height = self.get_latest_canonical_entry().height;
let migration_depth = self.migration_depth;
// 마이그레이션 깊이를 고려한 처리 가능 여부 판단
block_height <= latest_canonical_height + migration_depth as u64
}
pub fn add_block(
&mut self,
block: &IrysBlockHeader,
commitment_snapshot: Arc<CommitmentSnapshot>,
epoch_snapshot: Arc<EpochSnapshot>,
ema_snapshot: Arc<EmaSnapshot>,
) -> Result<(), BlockTreeError> {
// 블록을 트리에 추가하고 필요시 재구성 수행
self.blocks.insert(block.block_hash, block.clone());
// 새로운 블록이 더 긴 체인을 만드는지 확인
if self.is_longer_chain(block) {
self.reorganize_to_new_tip(block.block_hash)?;
}
```

```
Ok(())
}
}
```

실행 페이로드 관리

1. 페이로드 캐시 시스템

```
pub struct ExecutionPayloadCache {
/// 로컬 저장된 페이로드 캐시
local_cache: Arc<RwLock<LruCache<B256, SealedBlock>>>,
/// 대기 중인 페이로드 요청
waiting_for_payload: Arc<RwLock<HashSet<B256>>>,
/// Reth 페이로드 제공자
reth_payload_provider: RethBlockProvider,
}

impl ExecutionPayloadCache {
pub async fn wait_for_sealed_block(
&self,
evm_block_hash: &B256,
use_trusted_peers_only: bool,
) -> Option<SealedBlock> {
```

```
// 로컬 캐시에서 먼저 확인
if  let Some(sealed_block) = self.get_locally_stored_sealed_block(evm_block_hash
return Some(sealed_block);
}


// 대기 목록에 추가
self.waiting_for_payload.write().await.insert(*evm_block_hash);


// 네트워크에서 페이로드 요청
match  self.gossip_client.pull_payload_from_network(
*evm_block_hash,
use_trusted_peers_only,
&self.peer_list,
).await {
Ok((_, block)) => {
let  sealed_block = block.seal_slow();
self.add_payload_to_cache(sealed_block.clone()).await;
Some(sealed_block)
}
Err(err) => {
error!("Failed to fetch execution payload: {:?}", err);
None
}
```

```
}

}

}
```

2. 페이로드 검증 및 제출

```
async fn validate_and_submit_reth_payload(

&self,

block_header: &IrysBlockHeader,

reth_service: Option<Addr<RethServiceActor>>,

) -> Result<(), BlockPoolError> {

let adapter = self

.execution_payload_provider

.reth_payload_provider

.as_irys_reth_adapter()

.ok_or(BlockPoolError::OtherInternal("Reth provider not set".into()))?;


// 부모 에포크 스냅샷 가져오기

let parent_epoch_snapshot = self

.block_status_provider

.block_tree_read_guard()

.read()

.get_epoch_snapshot(&block_header.previous_block_hash)
```

```
.ok_or_else(|| BlockPoolError::OtherInternal("Parent epoch snapshot not found".i


// 섀도우 트랜잭션 검증
shadow_transactions_are_valid(
&self.config,
&self.service_senders,
block_header,
adapter,
&self.db,
self.execution_payload_provider.clone(),
parent_epoch_snapshot,
block_index,
).await?;


// Reth 서비스에 포크 선택 업데이트 전송
if  let Some(reth_service) = reth_service {
reth_service
.send(ForkChoiceUpdateMessage {
head_hash: BlockHashType::Irys(block_header.block_hash),
confirmed_hash: None,
finalized_hash: None,
})
.await??;
```

```
}


// 처리 완료 후 캐시에서 페이로드 제거
self.execution_payload_provider
.remove_payload_from_cache(&block_header.evm_block_hash)
.await;


Ok(())
}
```

성능 최적화 및 복구 메커니즘

1. 배치 처리 최적화

```
const BLOCK_BATCH_SIZE: usize = 10;
const PERIODIC_SYNC_CHECK_INTERVAL_SECS: u64 = 30;


// 배치 단위로 블록 인덱스 요청
let  block_index = get_block_index(
peer_list,
&api_client,
sync_state.sync_target_height(),
BLOCK_BATCH_SIZE,
```

```
5,
fetch_index_from_trusted_peer,
).await?;
```

2. 큐 관리 및 백프레셔

```
// 블록 큐가 가득 찬 경우 대기
if sync_state.is_queue_full() {
debug!("Block queue is full, waiting for empty slot");
// 재시도 로직
let retry_attempts = 3;
let mut wait_success = false;

for attempt in 1..=retry_attempts {
match sync_state.wait_for_an_empty_queue_slot().await {
Ok(()) => {
wait_success = true;
break;
}
Err(_) => {
warn!("Timeout waiting for queue slot, attempt {}", attempt);
// 처리를 트리거하기 위해 블록 재요청
let retry_height = sync_state.highest_processed_block() + 1;
```

```
self.request_retry_block(retry_height).await;
tokio::time::sleep(Duration::from_millis(1000)).await;
}
}
}

if !wait_success {
return Err(ChainSyncError::Internal("Queue slot timeout".to_string()));
}
}
```

3. 누락된 페이로드 복구

```
pub  async  fn  repair_missing_payloads_if_any(
&self,
reth_service: Option<Addr<RethServiceActor>>,
) -> Result<(), BlockPoolError> {
let Some(latest_block) = self.block_status_provider.latest_block_in_index() else
return Ok(());
};

let  mut  block_hash = latest_block.block_hash;
let  mut  blocks_with_missing_payloads = vec![];
```

```
// 누락된 페이로드가 있는 블록들 찾기
loop {
let  block = self.get_block_data(&block_hash).await?.ok_or(
BlockPoolError::PreviousBlockNotFound(block_hash)
)?;

let  payload_exists = self
.execution_payload_provider
.get_locally_stored_sealed_block(&block.evm_block_hash)
.await
.is_some();

if  payload_exists || block.height <= 1 {
break;
}

block_hash = block.previous_block_hash;
blocks_with_missing_payloads.push(block);
}

// 가장 오래된 블록부터 페이로드 복구
while  let Some(block) = blocks_with_missing_payloads.pop() {
```

```
debug!("Repairing missing payload for block {:?}", block.block_hash);
self.validate_and_submit_reth_payload(&block, reth_service.clone()).await?;
}

Ok(())
}
```

모니터링 및 디버깅

1. 동기화 상태 추적

```
impl ChainSyncState {
pub fn is_syncing(&self) -> bool {
self.is_syncing.load(Ordering::Relaxed)
}
pub fn sync_target_height(&self) -> usize {
self.sync_target_height.load(Ordering::Relaxed)
}
pub fn highest_processed_block(&self) -> usize {
self.highest_processed_block.load(Ordering::Relaxed)
}
pub async fn wait_for_sync(&self) {
while self.is_syncing() {
```

```
tokio::time::sleep(Duration::from_millis(100)).await;
}
}
}
```

2. 성능 메트릭스

```
// 동기화 진행률 추적
let progress = (sync_state.highest_processed_block() as f64 /
sync_state.sync_target_height() as f64) * 100.0;

debug!("Sync progress: {:.2}% ({}/{})",
progress,
sync_state.highest_processed_block(),
sync_state.sync_target_height());

// 블록 처리 시간 측정
let start_time = std::time::Instant::now();
self.block_discovery.handle_block(block_header, skip_validation).await?;
let processing_time = start_time.elapsed();

debug!("Block processing took: {:?}", processing_time);
```

3. 오류 복구 및 재시도

```
// 네트워크 오류 시 재시도 로직
for attempt in 1..=max_retries {
match api_client.get_block_index(peer.address.api, query).await {
Ok(index) => return Ok(index),
Err(err) => {
warn!("Attempt {}/{} failed: {}", attempt, max_retries, err);
if attempt < max_retries {
tokio::time::sleep(Duration::from_millis(1000 * attempt as u64)).await;
}
}
}
}
```

이 체인 동기화 및 블록 전파 시스템은 Irys 체인의 분산 특성을 지원하며, 높은 처리량과 안정성을 제공하는 동시에 네트워크 파티션과 같은 예외 상황에서도 견고한 복구 능력을 보장합니다.